

COMP 3133 – Full Stack Development II

Lab 07 – Angular

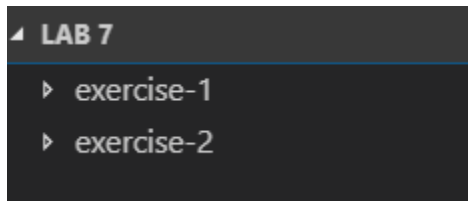
Topics:

- Express Generator + Router
- Angular First Application

Developer Note:

When working on your exercises, please create separate folder for your work. This way you won't be putting all your code in the same file, which can pollute the global name space. In short, it will prevent you from overwriting your own work and causing your code to compile incorrectly.

Organize your folder structure in this way.



Exercise #1 – Express Generator

1. Create a folder named Lab07_StudentID
2. Open a command prompt create a directory for **exercise-1**
3. Open Visual Studio Code and open the folder **exercise-1**
4. Run the following to install the express application generator

```
npm install express-generator -g
```

5. The following creates an Express app named **myapp**. The app will be created in a folder named **myapp** in the current working directory and the view engine will be set to [Pug](#)

```
express --view=pug myapp
```

The result will be the following file skeleton application structure being created in **myapp**.

```
create : myapp\  
create : myapp\public\  
create : myapp\public\javascripts\  
create : myapp\public\images\  
create : myapp\public\stylesheets\  
create : myapp\public\stylesheets\style.css  
create : myapp\routes\  
create : myapp\routes\index.js  
create : myapp\routes\users.js  
create : myapp\views\  
create : myapp\views\error.pug  
create : myapp\views\index.pug  
create : myapp\views\layout.pug  
create : myapp\app.js  
create : myapp\package.json  
create : myapp\bin\  
create : myapp\bin\www
```

6. Switch to **myapp** directory and then install the application dependencies

```
change directory:  
> cd myapp  
  
install dependencies:  
> npm install
```

7. Navigate to the **package.json** file and review the following scripts section. A CLI **start** script command has been generated so that node will execute a file from the bin/www folder.

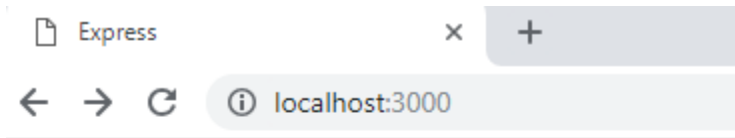
<https://docs.npmjs.com/cli/run-script>

```
"scripts": {  
  "start": "node ./bin/www"  
},
```

8. Run the **npm start** CLI command to start the web server on port 3000

```
npm start
```

9. Navigate the browser to the following URL and view the response from node.



Express

Welcome to Express

Exercise #2 – Routes + Body Parser

1. In the same **exercise-1** directory, stop the running node application and install the following body-parser middleware.

<https://www.npmjs.com/package/body-parser>

```
npm install body-parser --save
```

2. In the directory structure navigate to the **routes** folder.

```
└ routes
  JS index.js
  JS users.js
```

3. In the **users.js file** add the following code to include the **body-parser** middleware and set up the router to use the body-parser. Modify the code to reflect the following:

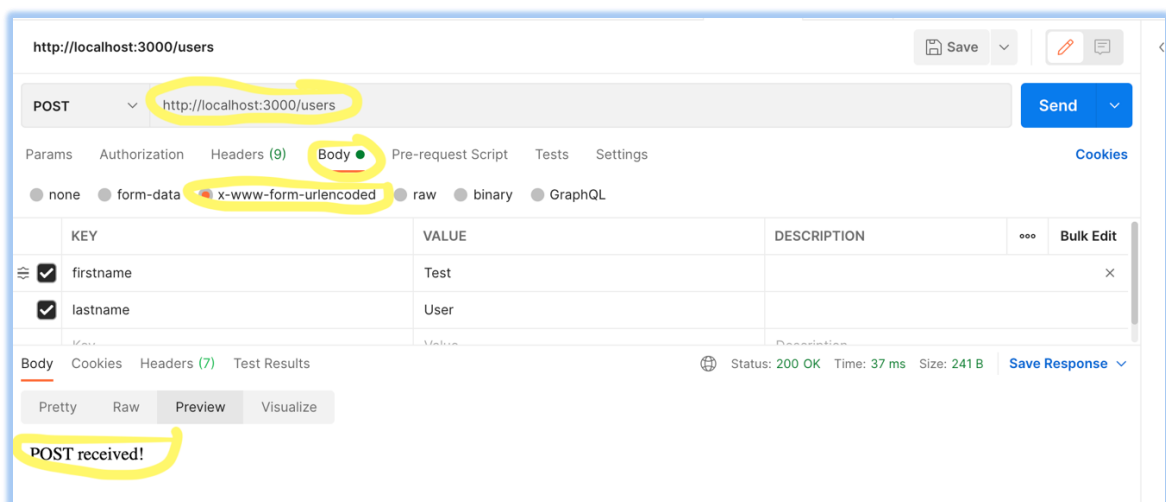
```

1 var express = require('express');
2 var bodyParser = require('body-parser')
3
4 var router = express.Router();
5
6 /* GET users listing. */
7 router.get('/', function(req, res, next) {
8   res.send('respond with a resource');
9 });
10
11 router.post('/', function(req, res, next) {
12   console.log(`First Name : ${req.body.firstname}`)
13   console.log(`Last Name : ${req.body.lastname}`)
14   res.send('POST received!');
15 });
16
17 router.use(bodyParser.urlencoded({extended: true}));
18
19 module.exports = router;

```

4. Remember to stop the running application and then rebuild the application with **node app.js** and then run the application with **npm start**

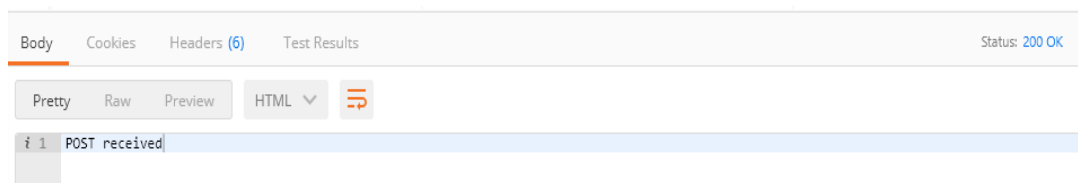
5. Using POST API Testing Tool build some post parameters and the following POST URL



6. Write a route in the **users.js** that handles the POST submission and uses **body-parser** middleware to pull the request parameters from the form and display them in the console log.

```
First name: Test  
Last name: User  
POST /users 200 3.995 ms - 13
```

The response sent to client i.e. POSTMAN from the user route should be the following 'POST received!' message:



Exercise #3 – Angular CLI + First Application

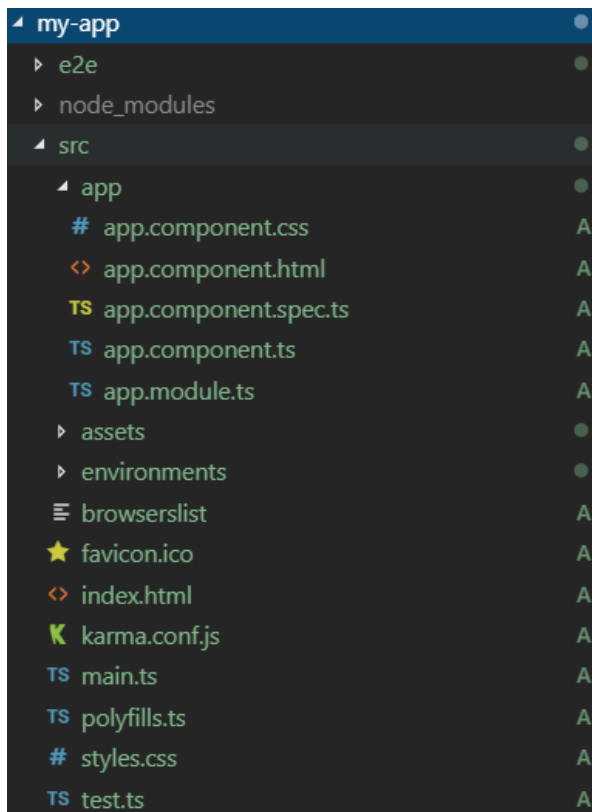
1. Open a command prompt create a directory for **exercise-2**
2. Open Visual Studio Code and open the folder **exercise-2**
3. At the command line using **npm** to install the **Angular CLI** globally.

```
npm install -g @angular/cli
```

4. Create a new project and default angular application named **my-app** by running the following CLI command.

```
ng new my-app
```

The following angular application file structure will be generated.



5. Navigated to the **my-app** directory and launch the application using the CLI commands

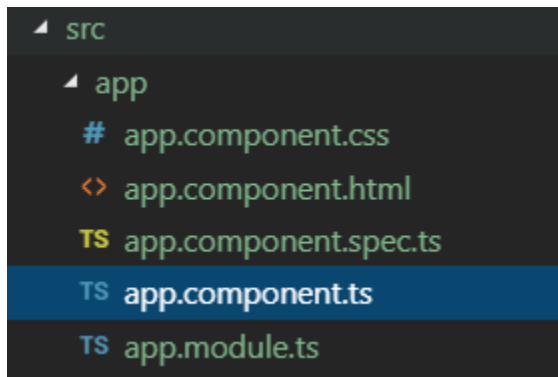
```
ng serve --open
```

6. Once the Angular development server is loaded it will launch the site on localhost with the following page.

Welcome to my-app!



7. Navigate to the root component **app.component.ts** and edit your first typescript file.



8. Change the title of the Angular to the following:

```
export class AppComponent {  
  title = 'Hello World!';  
}
```

The server will automatically track the changes and update the browser page to the following:

Welcome to Hello World!!



Submission

- **Screenshot(s)** of appropriate output. You may add all the screenshots in word document and upload it. ***[Do not put screenshots in project/zip file]***.
- Create a **zip file** of your entire project folder and submit.
- Your **GitHub** project link.